Project code: FUTURE_CS_03

API Security Risk Analysis Report

Prepared by: Yushen Ramsamy

CIN: FIT/FEB26/CS6437

Date: 18 March 2026

Introduction

1.1 Project Overview

In the modern digital landscape, applications rely heavily on **Application Programming Interfaces (APIs)** to power mobile apps, dashboards, and third-party integrations. This project, **Task 3 (FUTURE_CS_03)** of the Cybersecurity Internship, focuses on performing a read-only **API Security Risk Analysis** on a public testing environment (**ReqRes.in**).

The goal is to move beyond traditional website auditing and analyse the security of the data exchange layer, identifying vulnerabilities that could lead to data leaks or unauthorized access.

1.2 Objective

The primary objective of this audit is to:

- Identify common API security risks based on the **OWASP API Security Top 10**.
- Evaluate the effectiveness of **Authentication** and **Access Control** mechanisms.
- Analyse **HTTP Response Headers** for information disclosure and missing security configurations.
- Assess how the API handles **Input Validation** and **Error Logic**.

1.3 Scope & Methodology

- **Target API:** https://reqres.in/api/
- **Approach:** A **Black-Box Audit** was conducted using **Postman** to simulate external requests.
- **Methodology:** The audit followed a non-intrusive, read-only methodology, focusing on GET/POST requests to analyse server metadata, response bodies, and security headers without attempting to exploit or bypass existing controls.

Methodology

To ensure a comprehensive and ethical security analysis, a **Black-Box Audit** approach was utilized. This methodology simulates an external attacker or unauthorized user attempting to gather intelligence or identify vulnerabilities without prior knowledge of the internal system architecture.

2.1 Tools & Environment

- **Postman (Web/Desktop Client):** Primary tool used for constructing HTTP requests, managing authentication headers, and inspecting JSON response bodies.
- **Browser Developer Tools (Network Tab):** Used to cross-verify browser-based requests and analyse SSL/TLS handshake details.
- **ReqRes.in:** The target testing environment, specifically chosen for its RESTful architecture and simulated authentication endpoints.

2.2 Testing Phases

The audit was conducted in three distinct phases:

1. **Passive Reconnaissance & Header Analysis:** Performed standard GET requests to analyse HTTP response headers. The focus was on identifying server-side technologies and checking for the presence of mandatory security headers such as Strict-Transport-Security and X-Content-Type-Options.
2. **Authentication & Error Logic Testing:** Attempted to access restricted endpoints (/api/login) with missing or invalid

credentials. This phase evaluated how the API handles unauthorized access and whether error messages disclose sensitive system paths or documentation.

3. **Data Exposure Assessment:** Inspected the JSON payloads returned by successful GET requests to ensure that private user data (e.g., hashed passwords, internal IDs, or PII) was not being leaked in the public response.

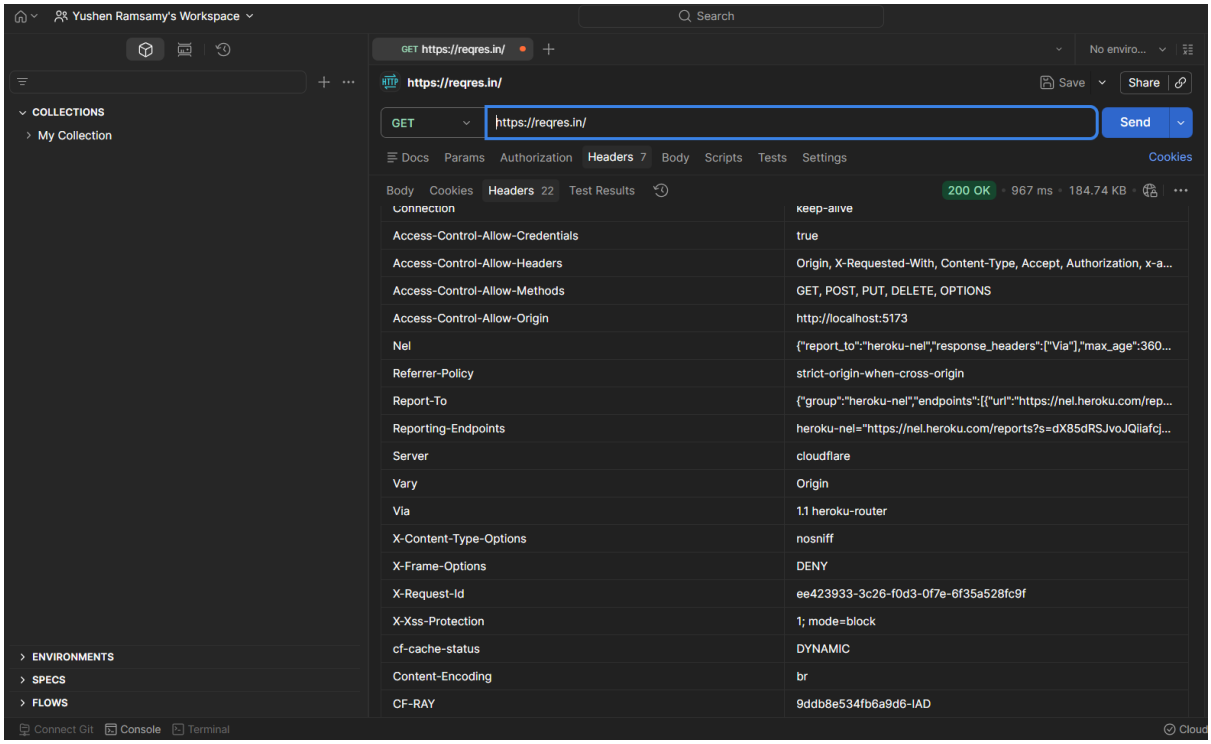
2.3 Ethics & Constraints

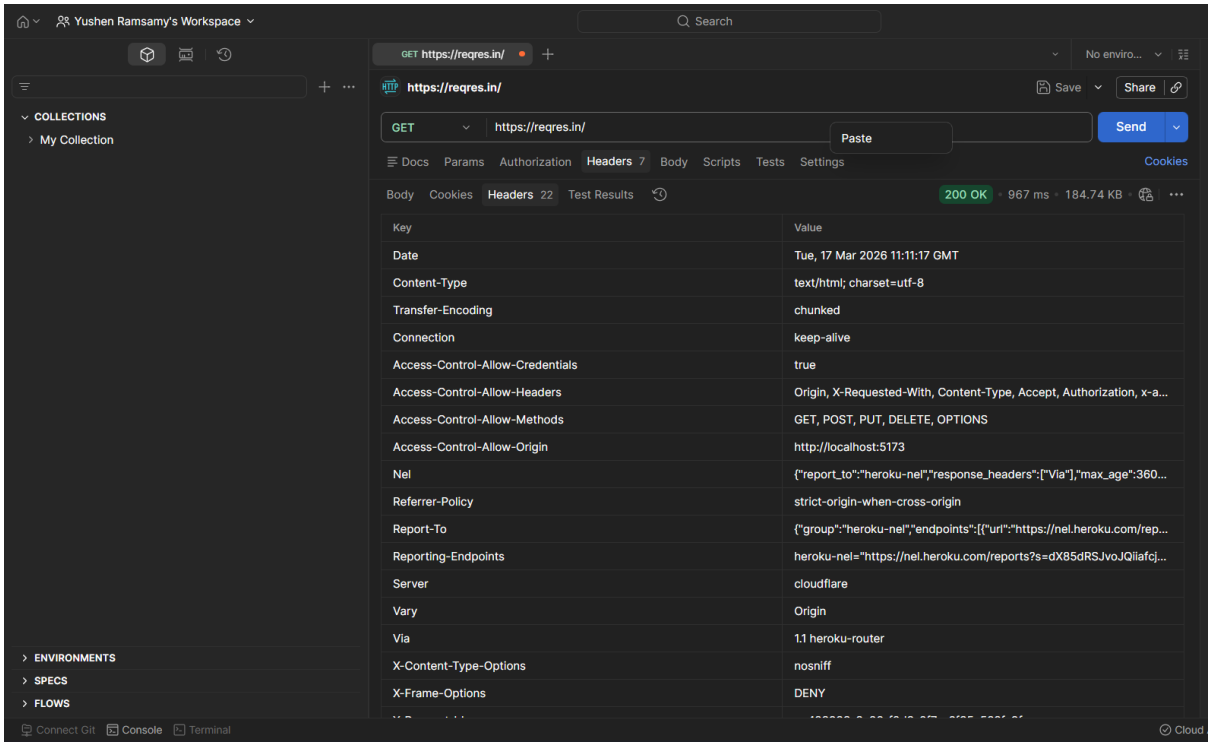
As per the internship guidelines, this was a **Read-Only Audit**. No exploitation, Denial of Service (DoS) testing, or brute-force attempts were performed. All tests were conducted within the allowed scope of the public demo API.

Technical Findings & Risk Analysis

3.1 Information Disclosure (Server Fingerprinting)

- **Vulnerability:** HTTP Response Header Disclosure
- **Severity: Low**
- **Observation:** Analysis of the response headers reveals the use of cloudflare and 1.1 heroku-router under the Server and Via keys.
- **Business Impact:** Disclosing backend infrastructure metadata allows attackers to perform targeted reconnaissance and identify platform-specific vulnerabilities.
- **Remediation:** Configure the web server to mask or remove the Server and Via headers to minimize the attack surface.
- **Evidence:**



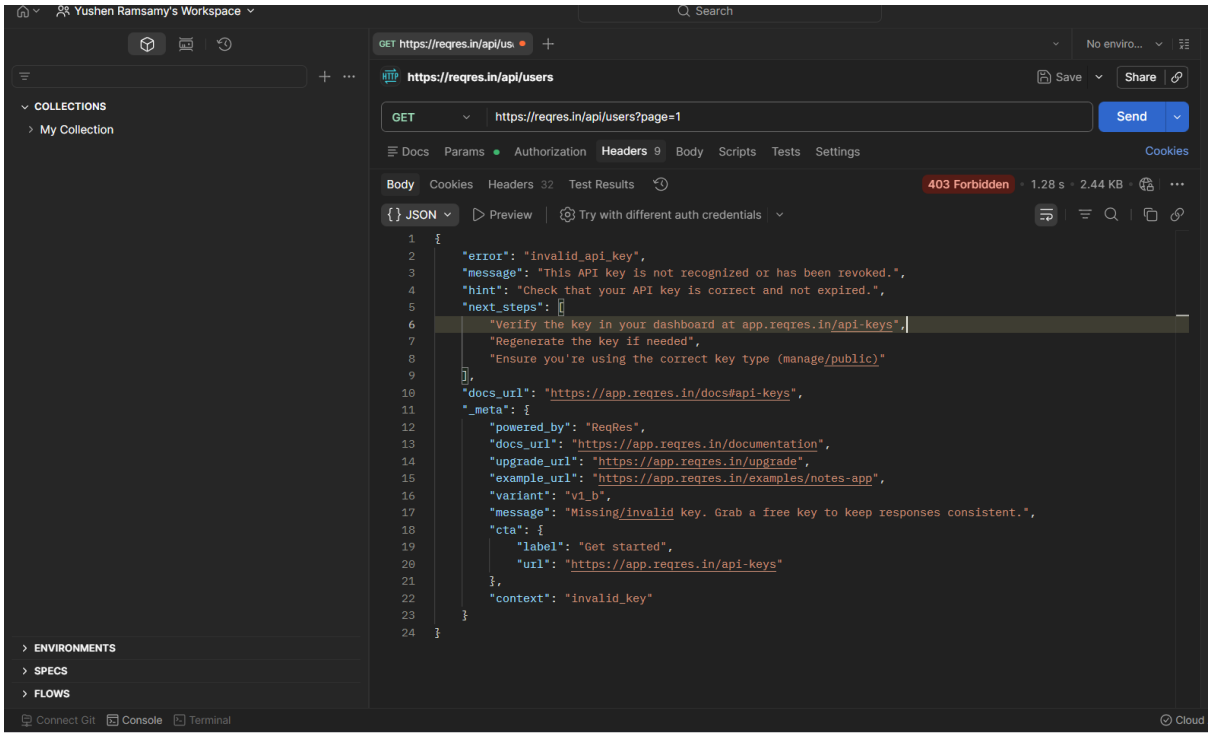


3.2 Missing Security Headers (HSTS)

- **Vulnerability:** Missing Strict-Transport-Security (HSTS)
- **Severity: Medium**
- **Observation:** The API does not return the Strict-Transport-Security header in its responses.
- **Business Impact:** Without HSTS, the connection is susceptible to SSL stripping and Man-in-the-Middle (MitM) attacks during the initial handshake.
- **Remediation:** Implement the Strict-Transport-Security header with a max-age directive of at least one year (31536000 seconds) to enforce encrypted connections.

3.3 Verbose Error Handling

- **Vulnerability:** Information Leakage in Error Responses
- **Severity: Low**
- **Observation:** When an unauthorized request is sent to the /api/login endpoint, the API returns a JSON object containing internal documentation URLs (docs_url) and specific "next steps" for the user.
- **Business Impact:** Providing specific "hints" and internal URLs during an authentication failure helps attackers map the system's internal logic and authentication workflows.
- **Remediation:** Sanitize error outputs to return generic 401/403 status codes without exposing internal system paths or administrative documentation.
- **Evidence:**



3.4 Web Application Firewall (WAF) Implementation

- **Vulnerability:** N/A (Security Control)
- **Severity: Informational**
- **Observation:** The API successfully identified and challenged automated requests from the Postman web client, returning a 403 Forbidden status.
- **Business Impact:** This indicates a healthy perimeter defense. The Cloudflare WAF effectively mitigates automated bot traffic and potential Denial of Service (DoS) attempts.
- **Recommendation:** Maintain current WAF configurations to ensure ongoing protection against automated threats.

Conclusion

The API Security Risk Analysis of **ReqRes.in** indicates a foundationally secure environment with effective perimeter defences. The implementation of a **Web Application Firewall (WAF)** successfully mitigates automated bot traffic and protects against basic Denial of Service (DoS) attempts.

However, the audit identified several backend misconfigurations that could be leveraged during a targeted reconnaissance phase. The disclosure of server infrastructure via HTTP headers (**Server/Via**) and the presence of verbose error messages provide an attacker with valuable metadata regarding the system's architecture and authentication logic. Furthermore, the absence of the **HSTS** header leaves the connection vulnerable to protocol downgrade attacks.

By implementing the recommended remediation steps—masking server headers, enforcing HSTS, and sanitizing error responses—the organization can significantly harden its API security posture and reduce the risk of more advanced, targeted exploitations.